

Tribhuvan University

Faculty of Humanities and Social Science

Samarpan Academy

A PROJECT REPORT

ON

"RESTROVIBES: A RESTAURANT BOOKING & SLOT- LOTTERY PLATFORM"

Submitted to

Department of Computer Application

Samarpan Academy

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

[Student Full Name]

TU Registration number: [TU-Reg-No]

[Month Year]

Under the Supervision of

[Supervisor Full Name]

Tribhuvan University

Faculty of Humanities and Social Science

Samarpan Academy

SUPERVISOR'S RECOMMENDATION

I hereby recommend that the project prepared under my supervision by [Student Full Name], titled "RestroVibes: A Restaurant Booking & Slot-Lottery Platform", be accepted as fulfilling the requirement for the degree of Bachelor of Computer Application, and be forwarded for final evaluation.

SIGNATURE

[Supervisor Full Name]

SUPERVISOR

Faculty of Humanities and Social Science

Dhumbarahi, Kathmandu

Tribhuvan University

Faculty of Humanities and Social Science

Samarpan Academy

LETTER OF APPROVAL

We, the undersigned, have examined the project report titled "RestroVibes: A Restaurant Booking & Slot-Lottery Platform" prepared by [Student Full Name] for the partial fulfilment of the degree of Bachelor in Computer Application. In our assessment, the report meets the required standard of scope and quality and is hereby approved.

Signature of Supervisor

[Supervisor Full Name]

Samarpan Academy

Dhumbarahi, Kathmandu

Signature of HOD / Coordinator

[Coordinator Name]

Coordinator

Samarpan Academy

Signature of Internal Examiner

Signature of External Examiner

ABSTRACT

Unfilled restaurant reservations cause substantial revenue loss and leave dining capacity idle. RestroVibes tackles this by providing a platform where clients and restaurateurs coordinate table bookings through a transparent, data-driven process. Every reservation is checked against live seat and table capacity before it is committed, so the same slot can never be sold twice. Each client is assigned a reliability score computed from their past behaviour, and whenever several guests compete for one contested slot, a weighted lottery awards it fairly to the most flexible, loyal, and dependable candidate. Background jobs keep the system self-managed by auto-confirming quiet bookings, detecting overstays, and recording no-shows, while GPS-based search helps clients find nearby restaurants along with live occupancy and travel estimates.

The platform is implemented as a React single-page application talking to a Node.js and Express API backed by PostgreSQL through the Sequelize ORM. Communication is protected with a JSON Web Token access and refresh scheme over RESTful endpoints. The outcome is a responsive and modular product that centralises appointments, table capacity, reliability scoring, and customer relations in one coherent system.

Keywords: Restaurant Booking, React, Node.js, Express, PostgreSQL, Sequelize, REST API, JWT, Weighted Lottery, Reliability Scoring

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the generous support I received along the way. I am especially grateful to my supervisor, [Supervisor Full Name], whose thoughtful guidance, patient review, and steady encouragement shaped both the direction and the quality of this work. My family has been a constant source of strength, and my friends and classmates contributed through lively discussions and timely assistance whenever I encountered obstacles.

I also appreciate the college for providing access to its library and a supportive space for study and research. Finally, I thank the wider open-source community whose frameworks and tools formed the technical backbone of this system. To everyone who helped in large ways or small, I offer my sincere gratitude.

[Student Full Name]

LIST OF ABBREVIATION

API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
GPS	Global Positioning System
HTML	Hypertext Markup Language
JS	JavaScript
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENT	v
LIST OF FIGURES	vii
LIST OF TABLES	ix
LIST OF ABBREVIATION	x
Chapter 1: Introduction	1
Chapter 2: Background Study and Literature Review	4
Chapter 3: System Analysis and Design	5
Chapter 4: Implementation and Testing	21
Chapter 5: Conclusion and Future Recommendation	25
REFERENCES	26
APPENDICES	27

LIST OF FIGURES

Figure 1.1 Agile Methodology	3
Figure 3.1 Use case diagram	6
Figure 3.2 Gantt Chart	8
Figure 3.3 Class Diagram of RestroVibes	9
Figure 3.4 Sequence Diagram	11
Figure 3.5 Activity Diagram	13
Figure 3.6 Component Diagram	15
Figure 3.7 Deployment Diagram	16
Figure 3.8 Working Mechanism of Reliability Scoring and Lottery	18

LIST OF TABLES

Table 4.1 Implementation Detail of Modules	21
Table 4.2 Test Case 001 – Login Module	22
Table 4.3 Test Case 002 – Appointment Creation	23

Chapter 1: Introduction

1.1 Introduction

Dining out is one of the most personal and location-dependent services a business can offer, yet the technology behind booking a table has lagged far behind. Most restaurants still rely on phone calls, paper ledgers, or generic calendar tools that give neither the guest nor the owner a reliable picture of what is actually available. RestroVibes is a restaurant appointment booking platform built to close that gap, offering a single place where clients can find a nearby restaurant, check live availability, reserve a table, and a restaurateur can track every booking, enforce seat capacity, and assess the reliability of the people asking for tables.

The system supports three different kinds of users. Clients can search for restaurants and make reservations, while restaurateurs manage their services, tables, and seating capacity, and admins supervise the entire platform. Underneath this simple interaction model lies a careful set of rules that keep bookings honest: capacity is checked before any reservation is accepted; each client carries a reliability score that influences how their requests are treated; and contested slots are resolved through a weighted lottery rather than raw timing.

1.2 Problem Statement

The restaurant industry loses a meaningful share of its revenue to guests who never arrive. A no-show wastes a seat for an entire time window, and there is usually no chance to refill it. The reverse problem is equally disruptive: when several guests request the same slot, the restaurateur is forced to make a quick decision with almost no information about who is most likely to show up. Both problems spring from the same root cause — reservations are managed without live capacity data and without a consistent, fair policy for deciding between competing requests.

Existing tools that merely store bookings do not solve this, because they record intentions without checking feasibility. What is missing is a system that validates each booking against real capacity, learns from guests' past behaviour, and applies an objective rule whenever demand exceeds supply. RestroVibes was designed specifically to supply those missing pieces.

1.3 Objectives

The following objectives guided the design and development of the system:

- to offer a unified place where clients and restaurateurs can manage reservations end to end;
- to validate every booking against live seat and table capacity before it is committed;
- to derive a reliability score for each client from their booking history;
- to allocate contested slots fairly through a weighted lottery;

- to automate routine decisions such as accepting quiet bookings, detecting overstays, and recording no-shows;
- to help clients discover nearby restaurants together with live occupancy and travel time.

1.4 Scope and Limitation

This part of the report draws the boundary around what the system does and what it deliberately leaves out, so that expectations about the platform remain realistic.

1.4.1 Scope of System

RestroVibes covers the whole journey of a table reservation. Clients can register, locate restaurants on a map, and request bookings, while restaurateurs maintain services and tables, set their seating capacity, and confirm or reject reservations. Behind those screens, the platform computes reliability scores, runs the weighted lottery, and keeps the appointment lifecycle in order through background workers. It also supports a two-way rating system so that both sides of the table can give feedback about a completed visit.

1.4.2 Limitation of Existing System

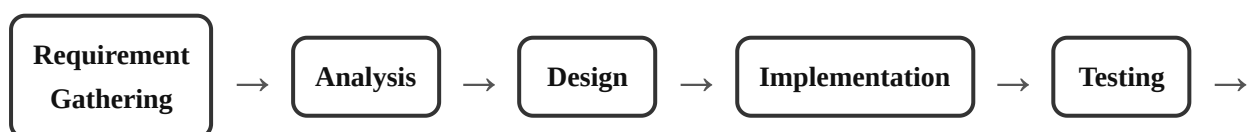
While the platform covers a great deal of ground, a few boundaries were left in place:

- distances are measured as straight-line, great-circle distances rather than road-based routes;
- payments are recorded for reporting purposes but are not actually processed;
- reliability scores depend on accurate and consistent booking history; and
- a single organisational structure is supported, so linked branches of one restaurant are not yet handled together.

1.5 Development Methodology

The project followed an iterative, feedback-driven development process. Rather than attempting to build everything in one pass, the work moved through repeated cycles in which a small set of features was designed, coded, reviewed, and refined before the next cycle began. This Agile approach made it easy to adjust priorities as requirements became clearer and kept each module testable from an early stage. The technology choices — React for the interface, Node.js with Express for the backend, and PostgreSQL for storage — were selected because they are modular, well documented, and scalable enough to accommodate future growth.

Figure 1.1 Agile Methodology



1.6 Report Organization

The remainder of this report is arranged into four further chapters. Chapter 2 offers a brief survey of the domain and related work. Chapter 3 records the analysis of requirements and the design of the system, including its main diagrams. Chapter 4 describes how the system was implemented and how it was tested. Chapter 5 closes with a conclusion and a list of possible future directions.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Reservation management sits at the centre of a restaurant's daily operations. Keeping tables full while honouring every confirmed guest requires accurate, up-to-the-minute knowledge of what is free, and that knowledge must be shared between the person taking the booking and the person serving the table. Where this coordination is handled manually, double bookings and forgotten reservations are common, and there is little historical record to learn from when guests fail to arrive.

Over time, restaurants have moved from paper registers toward software that stores reservations, but most of those tools stop at simple record keeping. RestroVibes goes further by linking each reservation to real capacity, by tracking the reliability of each guest, and by applying a clear policy when demand exceeds what a single slot can hold. These additions turn booking from a bookkeeping task into a management decision supported by evidence.

2.2 Literature Review

A large body of prior work informs the design of the platform. Web-based reservation services, used widely in the hospitality sector, demonstrate how centralised booking with live table availability and customer feedback improves the experience for both diners and restaurants [1]. That thinking is echoed in the appointment and rating modules of this system.

Research on fair allocation of limited resources highlights the value of assigning guests to slots according to a measurable score rather than simple arrival order [2]. RestroVibes adopts that principle in its weighted lottery, where flexibility, loyalty, and reliability each contribute to a weight that determines who wins a contended slot. Studies of reputation and scoring systems also confirm that visible, consistent scoring shapes behaviour over time [3], which supports the decision to expose a client's reliability status to restaurateurs.

Finally, work on automated service-lifecycle management shows that predictable, supervised automation — such as accepting a quiet booking or flagging an overstaying guest — removes routine burden from staff and keeps operations consistent [4]. The background workers in this system operationalise that idea, leaving the restaurateur free to focus on guests rather than bookkeeping.

Chapter 3: System Analysis and Design

3.1 System Analysis

Analysis establishes what the platform must accomplish and the conditions under which it must operate. The requirements were gathered by considering the everyday workflow of a small restaurant and the frustrations of both restaurant owners and their customers, then translated into concrete functions and quality expectations.

3.1.1 Requirement Analysis

Requirements fall naturally into two groups: what the system must do, and how well it must do it.

- i. Functional Requirements
- ii. Non-Functional Requirements

i. Functional Requirements

The functional requirements describe the concrete actions the platform supports:

- **Management of Accounts** – create, log into, and administer client, restaurateur, and admin accounts.
- **Reservation Handling** – create, confirm, cancel, complete, and extend bookings.
- **Capacity Checks** – verify seat and table availability before accepting a request.
- **Reliability Accounting** – compute and maintain each client's reliability score.
- **Slot Dispute Resolution** – resolve contested slots using a weighted lottery.
- **Map-Based Discovery** – show nearby restaurants with occupancy and travel time.
- **Two-Way Feedback** – allow clients and restaurateurs to rate one another.
- **Supervision and Evidence** – provide dashboards, risk profiles, and demand summaries.

ii. Non-Functional Requirements

The non-functional requirements describe how the platform should behave:

- **Responsiveness** – availability and bookings update without noticeable delay.
- **Growth** – the design must tolerate a growing number of clients and restaurants.
- **Safety** – endpoints are protected with token-based authentication.
- **Stability** – graceful error handling and reliable data storage.
- **Ease of Use** – a clean, responsive interface that requires little training.



Figure 3.1 Use case diagram

The use-case diagram above brings together the three actors and the main actions they perform. A client registers or logs in, searches for restaurants, makes reservations, and may enter

a lottery when a slot is contested. A restaurateur manages offerings and tables, sets capacity, confirms or rejects bookings, marks arrivals and no-shows, and reviews client risk. An admin oversees users, restaurants, and bookings, and can resolve a lottery manually or recompute reliability scores.

3.1.2 Feasibility Study

i. Technical Feasibility

The chosen tools are mature and widely documented, which greatly reduces technical risk. React provides a responsive interface, Node.js and Express offer a lightweight and high-performance backend, and PostgreSQL supplies a robust relational store with support for transactions and enumerations. Everything is packaged with Docker so that the same configuration runs consistently on any machine.

ii. Operational Feasibility

The platform addresses real, day-to-day concerns of a restaurant: preventing double bookings, limiting no-shows, dealing fairly with competing requests, and keeping capacity visible at all times. Automating the routine parts of the booking lifecycle also lightens the load on staff, making the system easy to adopt in practice.

iii. Economic Feasibility

Because the entire stack is open source, development and licensing costs remain low. Deployment through containerisation keeps hosting predictable, and a modular codebase means that future changes can be made without rewriting large parts of the system.

iv. Schedule Feasibility

An iterative plan kept the project on track. The Gantt chart in Figure 3.2 shows the main phases and their expected durations.

Requirements and analysis spanned two weeks, followed by two weeks of design. Backend and frontend were developed in parallel over roughly three weeks each. A further two weeks covered unit, integration, and system testing, and a final week was set aside for deployment and initial feedback.

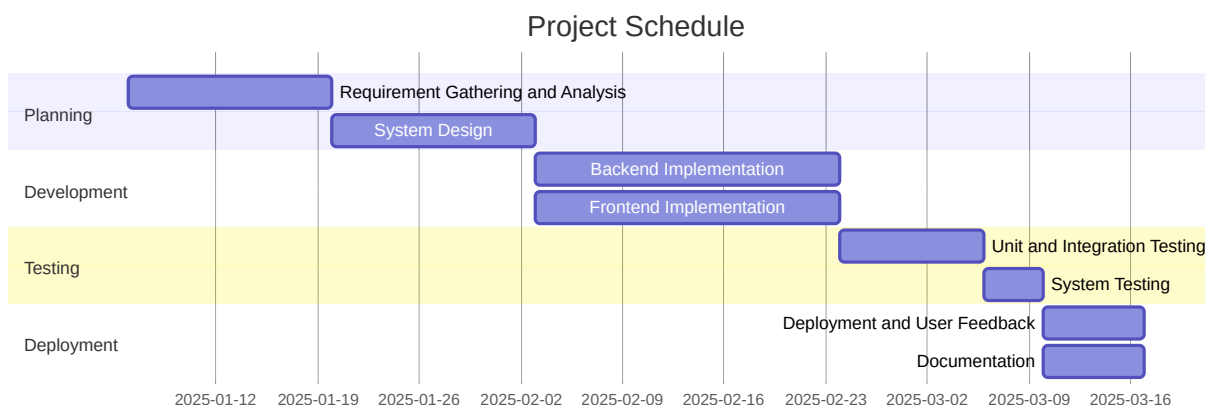


Figure 3.2 Gantt Chart

3.1.3 Object Modeling

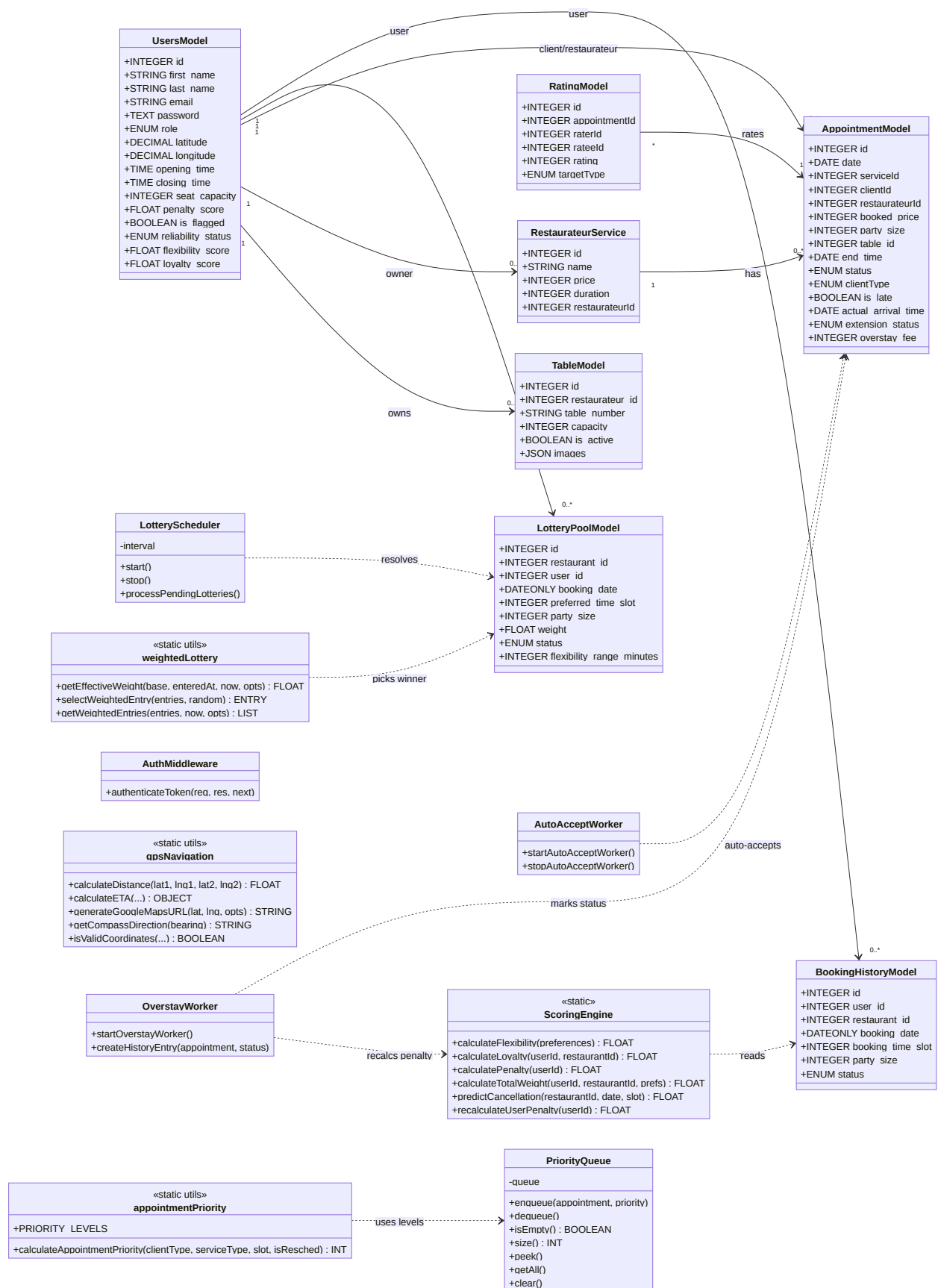


Figure 3.3 Class Diagram of RestroVibes

The class diagram expresses the structural backbone of the system. The central entity, *UsersModel*, stores identity, role, location, service hours, and seating capacity; a single user may

appear as client, restaurateur, or admin. Bookings are captured by *AppointmentModel*, which links a client and a restaurateur through a service and records date, party size, and status. A restaurateur owns *RestaurateurService* offerings and *TableModel* tables, while *RatingModel* and *BookingHistoryModel* preserve feedback and behavioural records. Contested slots are represented by *LotteryPoolModel*, which holds the weight assigned to each entry. Together these classes define how users, services, tables, ratings, history, and lottery entries relate to one another.

3.1.4 Dynamic Modeling

Dynamic modelling captures how the system behaves over time. The sequence diagram in Figure 3.4 walks through the two key workflows: making a reservation (including the lottery path) and the background handling of overstays.

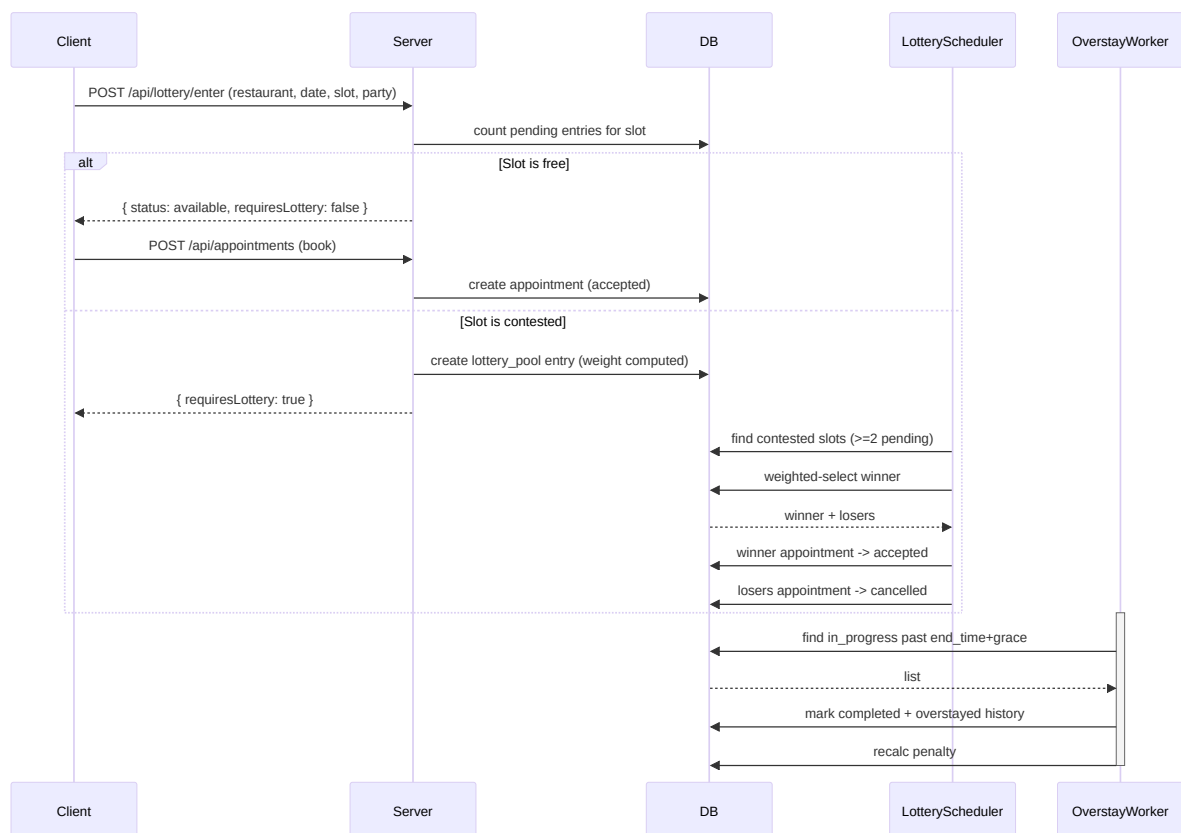


Figure 3.4 Sequence Diagram

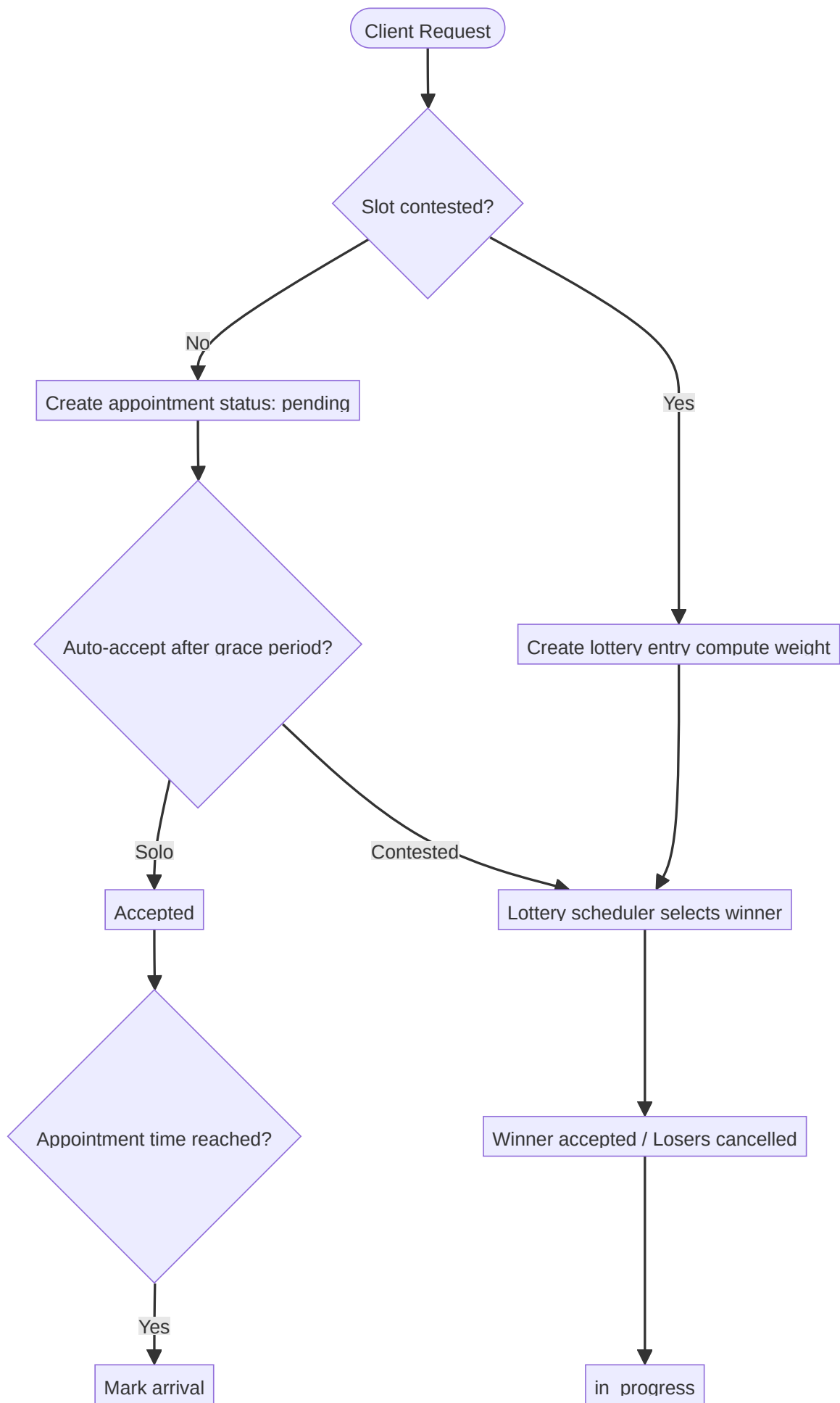
Booking and Lottery Flow

- A client sends a request for a restaurant, date, slot, and party size.
- The system counts the existing pending requests for that slot.
- If the slot is free, an appointment is created and accepted immediately.
- If it is contested, a lottery entry is stored with a computed weight.
- The lottery scheduler later selects a winner, accepting that booking and cancelling the rest.

Overstay Handling

- The overstay worker looks for active bookings whose end time plus a grace period has passed.
- It marks them completed, writes an overstay record, and refreshes the client's reliability score.

3.1.5 Process Modelling



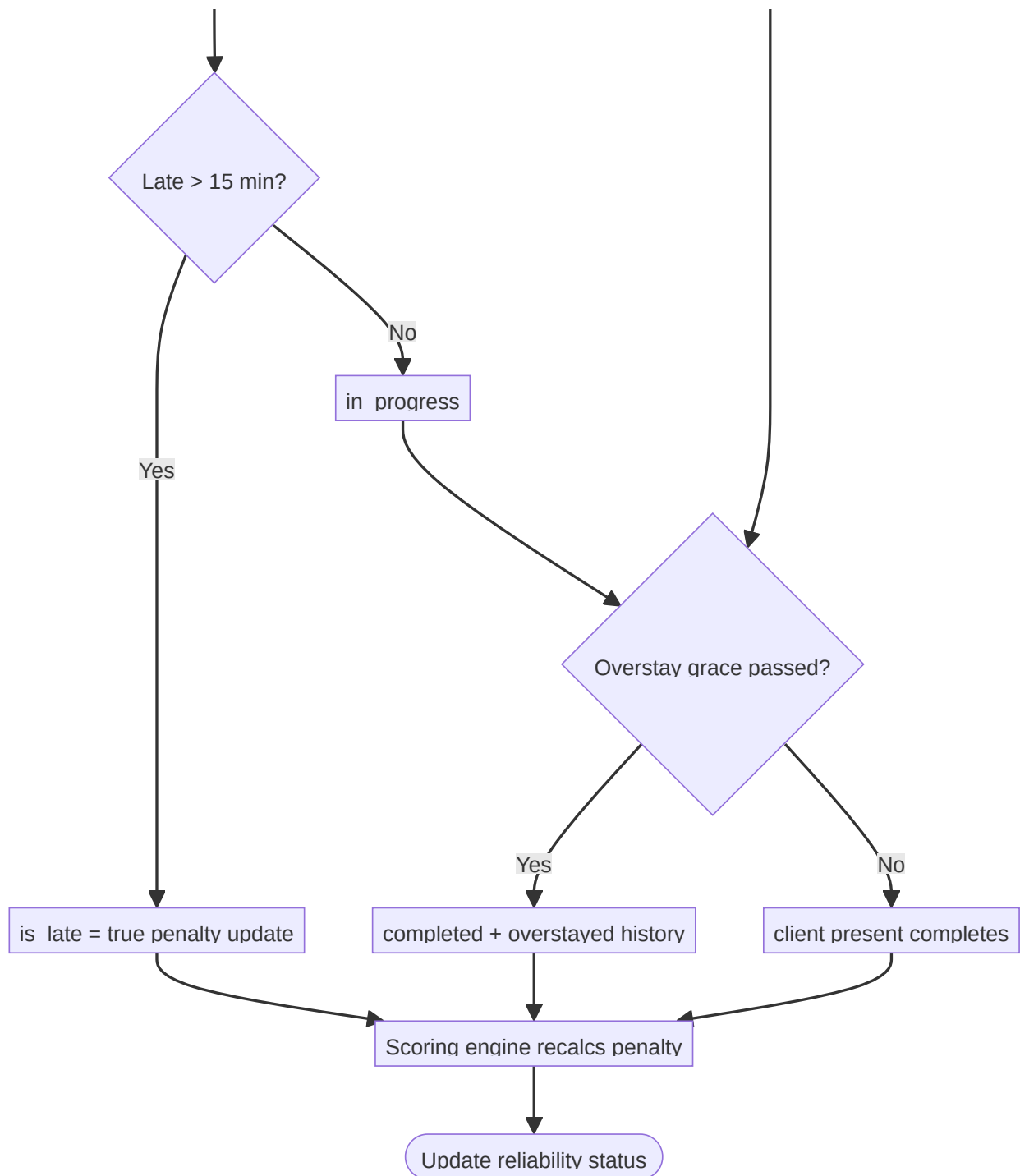


Figure 3.5 Activity Diagram

The activity diagram illustrates the life of a single reservation from request to settlement. A request is checked to see whether the slot is contested; an uncontested booking may be auto-accepted after a grace period, while a contested one passes through the lottery. Once the time arrives, the arrival is recorded, delayed arrivals are penalised, and, if a booking outlives its window, the overstay worker completes it. Every transition feeds the scoring engine, which keeps each client's reliability status current.

3.2 System Design

Design converts the analysed requirements into a concrete structure for the software: how it is organised internally and how it is deployed.

3.2.1 Component Diagram

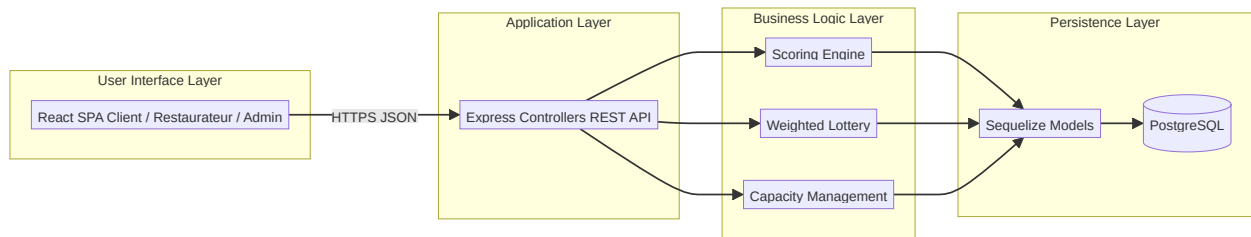


Figure 3.6 Component Diagram

The component diagram shows the platform separated into clear layers. At the top, a React interface lets each actor reach the system. Below it, Express controllers accept HTTP requests and route them into the business-logic layer, where the scoring engine, the weighted lottery, and capacity management do the real work. The persistence layer, realised with Sequelize models, translates those operations into SQL against the PostgreSQL database. This separation keeps each part of the system easy to test and change independently.

3.2.2 Deployment Diagram

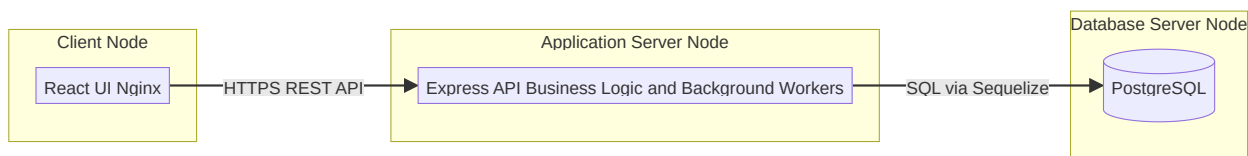


Figure 3.7 Deployment Diagram

The deployment diagram describes a three-tier arrangement. The client tier hosts the built React application behind Nginx, which also forwards API calls upstream. The application tier runs the Express backend, the business logic, and the background workers, talking to the data tier over the PostgreSQL driver. The database tier holds all records — users, appointments, services, tables, ratings, history, and lottery entries — and is reached only through the application tier, keeping the data layer shielded from direct client access.

3.3 Algorithm Details

Two algorithms lie at the heart of the platform: reliability scoring and the weighted lottery. Their working mechanism is summarised in Figure 3.8.

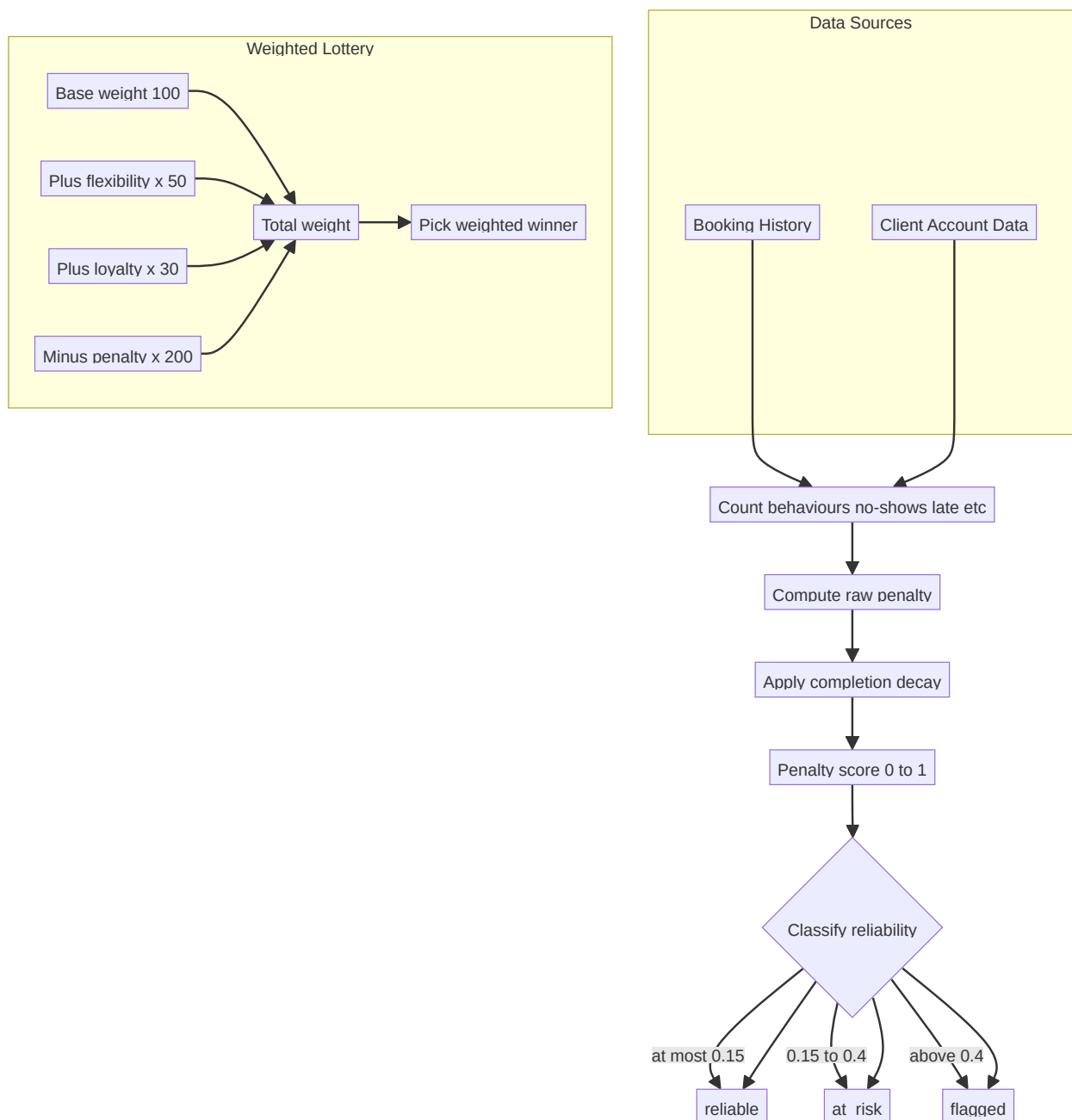


Figure 3.8 Working Mechanism of Reliability Scoring and the Weighted Lottery

Reliability Scoring

Each client's score begins with the behaviours observed on their booking history. A no-show, being late, cancelling late, or overstaying is assigned a weight reflecting its cost to the restaurant — a no-show is the most damaging, an overstay the least. Those weighted counts are combined and divided by the total number of bookings to obtain a raw penalty. A decay term, grown from the share of bookings that were completed, then softens the raw value so that consistent good behaviour reduces the penalty without erasing it entirely. The resulting score between zero and one is mapped to a reliability status: reliable, at risk, or flagged.

Weighted Lottery

When several clients want the same slot, each is given a weight starting from a base value. The weight increases with flexibility (the willingness to consider nearby times) and with loyalty (repeat business and account age), and decreases with the client's penalty score. An ageing factor is then added so that earlier entries carry a slight advantage. A weighted random selection then picks the winner, whose booking is accepted while the remaining requests are cancelled. This gives deserving, flexible, and reliable guests a fairer chance than a plain first-come-first-served rule would.

Location Discovery

Nearby-restaurant discovery relies on the Haversine formula to measure the great-circle distance between the client and each restaurant. Travel time is estimated at a modest average speed to keep the figure understandable. This means the "distance" shown is a straight-line measurement rather than a driving route, a deliberate simplification chosen for cost and simplicity.

Chapter 4: Implementation and Testing

4.1 Implementation

Implementation proceeded in short, focused cycles as planned in the methodology. At the start of each cycle a small set of features was chosen, coded against the design, and then reviewed before the next cycle began. This rhythm kept every module testable from the earliest stage and made it straightforward to respond when a requirement needed adjustment.

4.1.1 Tools Used

The implementation relied on a focused set of tools, each chosen for a specific role:

- **React** – builds the component-based user interface, giving a responsive and consistent experience.
- **Node.js / Express** – forms the API and the business logic behind it.
- **PostgreSQL** – stores the platform's relational data reliably.
- **Sequelize** – maps models to tables and manages associations and transactions.
- **Docker / Docker Compose** – packages the database, backend, and frontend for consistent deployment.
- **Leaflet and OpenStreetMap** – render the map used for nearby-restaurant discovery.
- **Nginx** – serves the built interface and proxies API calls in production.

4.1.2 Implementation Detail of Modules

Code	Module Name	Description
M1	Authentication Module	Registers and logs in users; issues and refreshes access and refresh tokens.
M2	Appointment Module	Creates, confirms, cancels, completes, extends, and tracks bookings.
M3	Capacity Module	Enforces seat and table capacity while a booking is made.
M4	Reliability Module	Computes and keeps each client's penalty and reliability status current.
M5	Lottery Module	Resolves contested slots among multiple clients.
M6	Discovery Module	Finds nearby restaurants with occupancy and travel estimates.
M7	Rating Module	Handles two-way ratings between clients and restaurateurs.
M8	Worker Modules	Auto-accept, overstay, and lottery scheduling background jobs.

Table 4.1 Implementation Detail of Modules

4.2 Testing

Testing ran alongside development so that problems were caught while the surrounding code was still fresh. Unit tests targeted individual functions, while system tests confirmed that whole workflows behaved correctly together.

4.2.1 Test Cases for Unit Testing

Project Name: RestroVibes — Restaurant Booking and Slot-Lottery Platform		
TEST CASE		
Test Case ID: Login_001	Test Design By: [Student Name]	
Module Name: Authentication (Login) Module	Test Design Date: [Date]	
Test Title: Login User	Test Executed By: [Student Name]	
Description: Verify that a valid user can sign in	Test execution date: [Date]	
Pre-Conditions: A registered user exists with the correct password		
Dependencies: Backend API reachable		
Step	Test Step	Expected Result
1	Enter email and password	Both fields accept input
2	Press the login button	The user signs in and lands on the dashboard with an access token

Table 4.2 Test Case 001 – Login Module

Post-conditions: The credentials are validated and the user is taken to the dashboard holding a valid session token.

4.2.2 Test Cases for System Testing

The system tests examined complete user journeys. The example below verifies the booking path from the client's side.

Project Name: RestroVibes
TEST CASE

Test Case ID: TC_booking_creation_001	Test Design By: [Student Name]	
Module Name: Appointment Creation	Test Design Date: [Date]	
Test Title: Create a New Booking	Test Executed By: [Student Name]	
Description: Verify a client can reserve a table	Test execution date: [Date]	
Pre-Conditions: The client is signed in and a restaurant is available		
Dependencies: Active API connection and free capacity		
Step	Test Step	Expected Result
1	Open the booking page	Service and slot selection are shown
2	Choose restaurant, date, and slot	Availability is displayed correctly
3	Enter a party size within capacity	The booking is accepted, or a lottery begins for a contested slot
4	Confirm the reservation	The appointment is created and appears in history

Table 4.3 Test Case 002 – Appointment Creation

Post-conditions: The new appointment is visible to the client and is counted in the restaurant's capacity calculations.

Chapter 5: Conclusion and Future Recommendation

5.1 Conclusion

RestroVibes demonstrates that a booking platform can do more than store reservations; it can actively protect a restaurant's capacity and pool. By checking every request against live seat and table availability, the platform removes the possibility of overbooking at the source. By scoring each client's reliability, it gives the restaurateur a clear, honest picture of the risk each request carries. And by resolving contested slots through a weighted lottery, it turns an often arbitrary decision into a fair, explainable rule.

The use of React, Node.js, Express, and PostgreSQL produced a responsive and dependable product, and the background workers keep the appointment lifecycle in order with little human input. Altogether, the platform improves operational efficiency, gives both sides of the reservation clearer information, and lays a solid foundation for further growth.

5.2 Future Enhancement

Several extensions could build on the current system in later releases:

- **Actual payments** – connect a payment gateway so deposits or cancellation fees can be charged automatically.
- **Road-based travel times** – replace straight-line distances with real routing from a mapping service.
- **A mobile application** – allow booking and management while on the move.
- **Automated alerts** – send confirmations, reminders, and no-show notices by email or SMS.
- **Predictive analytics** – forecast demand and flag likely cancellations before they happen.
- **Multi-branch support** – let a restaurant manage several locations from one account.

REFERENCES

1. Reserve and OpenTable. *Online reservation management for the hospitality sector*. Online documentation, accessed 2025.
2. Wu, B. *Fair allocation of limited service capacity*. Proceedings on resource allocation in service systems, 2018.
3. Resnick, P., and Zeckhauser, R. *Trust systems and reputational scoring*. 2002.
4. Nandakumar, D., and Priya, A. *Automated lifecycle management for service platforms*. 2019.

APPENDICES

Appendix A — Client Booking Interface

Figure A.1 Client dashboard and booking screen

[Place a screenshot of the client booking interface here.]

Appendix B — Restaurateur Dashboard

Figure B.1 Restaurateur appointments and risk-profile view

[Place a screenshot of the restaurateur dashboard here.]

Appendix C — Admin Panel

Figure C.1 Admin bookings and restaurant listing

[Place a screenshot of the admin panel here.]